Topic 3: Working with JSON Data

N8N Hands-On API Integration Course • Step-by-Step Practical Guide

> **Overview:** This step-by-step guide walks you through establishing a nested JSON mock data payload, writing dot-notation expressions to extract child object parameters, referencing array elements by index positions, mapping properties dynamically between node dependencies, and running a complete data transformation workflow in n8n.

Step 1: Load Nested JSON Payload into Workflow

We will start by creating a mock JSON structure representing an invoice receipt:

1. In your n8n workspace, create a new workflow canvas.
2. Add a **Manual Trigger** node.
3. Click the output connector on the Manual Trigger and add an **Edit Fields (Set)** node.
4. Rename this node to: `Mock Invoice Data`.
5. In the node configuration sidebar:
   - **Specify Fields:** Select `Using JSON` from the dropdown menu.
   - **JSON:** Paste the nested JSON schema:

```
{
  "receiptId": "REC-9988",
  "customer": {
    "firstName": "John",
    "lastName": "Doe",
    "contact": {
      "email": "johndoe@example.com"
    }
  },
  "purchasedItems": [
    { "name": "Wireless Mouse", "price": 25.99 },
    { "name": "Mechanical Keyboard", "price": 89.99 }
  ],
  "discountCode": null
}
```

6. Click the **Test step** button. The output panel on the right will display this JSON.

Step 2: Extract Nested Object Properties using Dot Notation

To query variables stored inside children objects, we traverse the hierarchy using dot-notation:

1. Add a new **Edit Fields (Set)** node connected to the output of `Mock Invoice Data`.
2. Rename this node to: `Extract Customer Data`.
3. In the node configurations:
   - **Specify Fields:** Select `Using Fields` (default).
   - Under **Fields to Add**, click **Add Field** → **String**:

- **Name:** `customerEmail`
- **Value:** Enter expression: `{{ $json.customer.contact.email }}` (preview: `"johndoe@example.com"`).

- Click **Add Field** → **String**:
  - **Name:** `fullName`
  - **Value:** Enter expression: `{{ $json.customer.firstName }} {{ $json.customer.lastName }}` (preview: `"John Doe"`).

4. Click **Test step** to verify the extracted fields appear in the output.

Step 3: Reference Array Elements by Index

To fetch variables stored inside a list, query them using 0-based indexes:

1. In the same `Extract Customer Data` node, click **Add Field** → **String**:
   - **Name:** `firstItemName`
   - **Value:** Enter expression: `{{ $json.purchasedItems[0].name }}` (preview: `"Wireless Mouse"`).
2. Click **Add Field** → **Number**:
   - **Name:** `firstItemPrice`
   - **Value:** Enter expression: `{{ $json.purchasedItems[0].price }}` (preview: `25.99`).
3. Click **Test step** and confirm output contains: `customerEmail`, `fullName`, `firstItemName`, and `firstItemPrice`.

Step 4: Map Multiple Node Values to an HTTP Payload

Now we will use these dynamically extracted variables to populate an outbound API request body:

1. Add an **HTTP Request** node connected to the output of `Extract Customer Data`.
2. In the node configurations:
   - **Method:** Select `POST`.
   - **URL:** `https://jsonplaceholder.typicode.com/posts`
   - **Send Body:** Toggle to `Yes`.
   - **Body Content Type:** `JSON`.
   - **Specify Body:** `Using JSON`.
   - **JSON:** Paste:

```
{
  "title": "New Purchase for {{ $json.fullName }}",
  "body": "Sent email invoice to {{ $json.customerEmail }}"
}
```

3. Click **Test step** and verify that n8n compiles the payload dynamically and receives an HTTP 201 Created response.

Step 5: Save and Execute the Full Workflow

1. Rename the workflow to: **N8N JSON Traversal Workflow**.
2. Click **Save** in the top-right corner.
3. Click **Test workflow** at the bottom. Confirm that all nodes execute successfully with green checkmarks.
